

Swift / SwiftUI 多任务与并发全景指南

涵盖：Thread、RunLoop、GCD (`DispatchQueue.main / global` / 自定义队列 / Group / Semaphore / Barrier / WorkItem / Source)、`OperationQueue`、Swift Concurrency (`async/await`、`Task`、`Task.detached`、`Task.immediate`、`async let`、`TaskGroup`、`actor`、`MainActor`、`Sendable`、`Continuation`、`AsyncStream`、`Executor`)、锁与同步原语 (`NSLock`、`OSAllocatedUnfairLock`、`Mutex`、`Atomic`)、Combine Scheduler，以及 SwiftUI 中的 `.task`、`TimelineView` 等。

目录

- 0. 全景图：各概念所在的层次
- 1. 基础概念
- 2. Thread
- 3. RunLoop
- 4. GCD (Grand Central Dispatch)
- 5. OperationQueue
- 6. Swift Concurrency
- 7. 锁与同步原语
- 8. Combine 中的 Scheduler
- 9. SwiftUI 中与并发相关的 API
- 10. 横向对比
- 11. 执行时机与滚动行为
- 12. GCD → Swift Concurrency 迁移对照
- 13. 常见陷阱
- 14. 选择指南
- 15. 一句话总结

0. 全景图：各概念所在的层次

SwiftUI 层	<code>.task</code> / <code>.task(id:)</code> / <code>.refreshable</code> / <code>TimelineView</code> View 默认 <code>@MainActor</code>
Swift Concurrency (语言级)	<code>async/await</code> · <code>Task</code> · <code>Task.detached</code> · <code>async let</code> · <code>TaskGroup</code> <code>actor</code> · <code>@MainActor</code> · <code>global actor</code> · <code>Sendable</code> <code>AsyncSequence/AsyncStream</code> · <code>Continuation</code> · <code>TaskLocal</code> — 执行器: <code>MainActor executor</code> / 协作式线程池 —
Combine	Scheduler: <code>DispatchQueue</code> / <code>RunLoop</code> / <code>OperationQueue</code> / <code>ImmediateScheduler</code>
Foundation	<code>OperationQueue</code> / <code>Operation</code> (基于 GCD 封装) <code>RunLoop</code> / <code>Timer</code> / <code>Thread</code>
GCD (libdispatch)	<code>DispatchQueue.main</code> / <code>.global(qos:)</code> / 自定义队列 <code>DispatchGroup</code> / <code>Semaphore</code> / <code>WorkItem</code> / <code>Source</code> / <code>barrier</code>
OS / 内核	<code>pthread</code> · 线程调度 · 锁 (<code>os_unfair_lock</code> 等)

关键关系：

- 主线程上跑着 `RunLoop.main`；`DispatchQueue.main` 的任务由主 `RunLoop` 排空执行；`MainActor` 的执行器底层就是提交到主队列。
- `DispatchQueue.global()` 和自定义队列由 GCD 管理的线程池执行，GCD 会在线程阻塞时额外创建线程。
- Swift Concurrency 中非主 actor 的任务跑在协作式线程池（cooperative thread pool）上，线程数 $\approx$ CPU 核心数，不会因为阻塞而补充线程。
- `OperationQueue` 是对 GCD 的面向对象封装，增加了依赖、取消、并发数控制。

1. 基础概念

概念	含义
进程 / 线程	线程是 CPU 调度的基本单位。iOS App 有一个主线程（UI 线程）和若干后台线程。
队列（Queue）	GCD 的任务容器，FIFO。开发者面向队列而不是线程编程，由系统决定使用哪个线程。
任务（Task）	Swift Concurrency 中的异步工作单元，可以挂起、恢复，可能在不同线程上恢复。
串行 vs 并发	串行：同一时刻只执行一个任务。并发：可同时执行多个任务。
同步 vs 异步	同步（sync）：调用方等待任务完成。异步（async）：提交后立即返回。
并发 vs 并行	并发是结构（多个任务交替推进）；并行是执行（多个核心同时执行）。
阻塞 vs 挂起	阻塞（block）：线程卡住什么也不做，占着线程。挂起（suspend, <code>await</code> ）：任务让出线程，线程可以去执行其他任务。
数据竞争（Data Race）	多个线程同时访问同一内存且至少一个是写，没有同步。Swift 6 在编译期消除。
竞态条件（Race Condition）	结果依赖于执行时序的逻辑错误。即使没有数据竞争也可能发生（如 actor 重入）。
QoS / 优先级	告诉系统任务的重要程度，影响调度与能耗。

2. Thread

2.1 是什么

`Thread`（`NSThread`）是对 POSIX 线程的封装，是最底层的多线程方式。

```
let thread = Thread {
    print("在新线程中: \(Thread.current)")
}
thread.name = "com.example.worker"
thread.qualityOfService = .utility
thread.start()

Thread.isMainThread           // 当前是否主线程
Thread.sleep(forTimeInterval: 1) // 阻塞当前线程
```

2.2 何时使用

- 几乎不需要在 App 代码中直接使用。
- 极少数场景：需要一个长期存在、带自己 `RunLoop` 的线程（例如某些旧的网络库、需要专用线程的 C 库）。

2.3 注意

- 在 Swift Concurrency 的 `async` 函数中，`Thread.current` 已被标为不可用（Swift 6），因为任务在 `await` 前后可能运行在不同线程上，"当前线程"不再有意义。判断主线程隔离应使用 `MainActor.assumeIsolated / MainActor.preconditionIsolated()`。

3. RunLoop

3.1 是什么

`RunLoop` 是线程的事件处理循环："等待事件 → 处理事件 → 休眠"。每个线程最多一个 `RunLoop`，只有主线程的 `RunLoop` 默认在运行（由 `UIApplicationMain / SwiftUI App` 启动）。

关键概念是 **Mode（模式）**：

Mode	说明
<code>.default</code>	默认模式，空闲时处于此模式
<code>.tracking (UITrackingRunLoopMode)</code>	用户拖动/滚动 <code>ScrollView</code> 时切换到此模式
<code>.common</code>	伪模式，一个"集合"，默认包含 <code>.default</code> 和 <code>.tracking</code>

一个输入源（如 `Timer`）只会在**它被注册的模式**下触发。

3.2 典型场景

```
// 1) SwiftUI 中的定时器
struct CountdownView: View {
    @State private var remaining = 60
    // in: .common — 滚动时也继续触发（若用 .default, 滚动时会暂停）
    private let timer = Timer.publish(every: 1, on: .main, in:
.common).autoconnect()

    var body: some View {
        Text("\(remaining)")
            .onReceive(timer) { _ in
                if remaining > 0 { remaining -= 1 }
            }
    }
}

// 2) Combine 切换到 RunLoop.main — 在 .default 模式执行，滚动时会被推迟
imagePublisher
    .receive(on: RunLoop.main)
    .sink { self.thumbnail = $0 }

// 3) 在指定 mode 下于下一次 RunLoop 迭代执行
RunLoop.main.perform(inModes: [.common]) { /* ... */ }

// 4) CADisplayLink (逐帧回调)
let link = CADisplayLink(target: self, selector: #selector(step))
link.add(to: .main, forMode: .common)
```

3.3 特点

- 最底层，没有编译期检查。
- 与滚动强相关：注册在 `.default` 的任务在滚动时**不执行**。
- 用于 Timer、DisplayLink、端口、`perform(_:with:afterDelay:)` 等依赖 RunLoop 的 API。

4. GCD（Grand Central Dispatch）

GCD 的核心思想：把任务（block）提交给队列，由系统管理线程池执行。

4.1 三种队列

队列	类型	执行线程	用途
<code>DispatchQueue.main</code>	串行	主线程	UI 更新
<code>DispatchQueue.global(qos:)</code>	并发（系统共享）	线程池中的后台线程	一次性的后台工作
<code>DispatchQueue(label:)</code>	串行（默认）	线程池中的后台线程	保护共享状态、按顺序执行
<code>DispatchQueue(label:, attributes: .concurrent)</code>	并发（私有）	线程池中的后台线程	读写锁（配合 barrier）

4.2 DispatchQueue.main

```
// 旧式回调切回主线程
URLSession.shared.dataTask(with: url) { data, _, _ in
    let image = data.flatMap(UIImage.init(data:))
    DispatchQueue.main.async { self.image = image }
}.resume()

// 延迟执行
DispatchQueue.main.asyncAfter(deadline: .now() + 2) { showToast = false }

// 推迟到下一轮 RunLoop（例如 onAppear 后设置焦点）
.onAppear { DispatchQueue.main.async { isFocused = true } }
```

- `async` 总是入队，等当前代码执行完之后才执行。
- 主队列由主 RunLoop 在 **common modes** 下排空，所以滚动期间也会执行。
- 在主线程调用 `DispatchQueue.main.sync` → 死锁。
- Swift 6 编译器会把传给 `DispatchQueue.main.async` 的闭包视为 `@MainActor` 隔离，可直接访问主 actor 状态。

4.3 DispatchQueue.global(qos:)

系统提供的共享并发队列，按 QoS 区分：

QoS	场景	对应 TaskPriority
<code>.userInteractive</code>	动画、事件处理，需立即完成	—（主线程级别）
<code>.userInitiated</code>	用户点击后等待的结果（打开文档）	<code>.userInitiated</code> / <code>.high</code>
<code>.default</code>	默认值， <code>global()</code> 不传参时使用	<code>.medium</code>

QoS	场景	对应 TaskPriority
<code>.utility</code>	有进度条的长任务（下载、导入）	<code>.utility / .low</code>
<code>.background</code>	用户感知不到的任务（备份、索引、预取）	<code>.background</code>

```
DispatchQueue.global(qos: .userInitiated).async {
    let thumbnail = makeThumbnail(from: data) // 耗时工作
    DispatchQueue.main.async {
        self.thumbnail = thumbnail // 回到主线程更新 UI
    }
}
```

注意：

- global 队列是所有人共享的，不能用 barrier（在 global 队列上 barrier 等同普通 async），也不适合用来保护状态。
- 不要在 global 队列上大量提交会阻塞的任务——GCD 会为被阻塞的线程补充新线程，导致线程爆炸（thread explosion，上限约 64 个），引起内存与调度开销，甚至死锁。

4.4 自定义串行队列：保护共享状态

```
final class Cache: @unchecked Sendable {
    private var storage: [String: Data] = [:]
    private let queue = DispatchQueue(label: "com.example.cache") // 串行

    func value(for key: String) -> Data? {
        queue.sync { storage[key] }
    }

    func set(_ data: Data, for key: String) {
        queue.async { self.storage[key] = data }
    }
}
```

在 Swift Concurrency 中，这种模式通常由 `actor` 或 `Mutex` 取代。

4.5 并发队列 + barrier：读写锁

```
final class ThreadSafeDictionary<Key: Hashable, Value>: @unchecked Sendable {
    private var dict: [Key: Value] = [:]
    private let queue = DispatchQueue(label: "rw", attributes: .concurrent)

    subscript(key: Key) -> Value? {
        get { queue.sync { dict[key] } } // 多个读可并发
        set { queue.async(flags: .barrier) { self.dict[key] = newValue } } // 写独占
    }
}
```

4.6 DispatchGroup：等待一组任务完成

```

let group = DispatchGroup()
var results: [Data] = []
let lock = NSLock()

for url in urls {
    group.enter()
    URLSession.shared.dataTask(with: url) { data, _, _ in
        if let data { lock.withLock { results.append(data) } }
        group.leave()
    }.resume()
}

group.notify(queue: .main) {
    print("全部完成: \(results.count)")
}
// group.wait() 会阻塞当前线程，切勿在主线程调用

```

Swift Concurrency 对应: `withTaskGroup` / `async let`。

4.7 DispatchSemaphore: 限制并发数 / 同步等待

```

let semaphore = DispatchSemaphore(value: 3) // 最多 3 个并发
for item in items {
    DispatchQueue.global().async {
        semaphore.wait()
        defer { semaphore.signal() }
        process(item)
    }
}

```

⚠ 绝不能在 Swift Concurrency 的 `async` 上下文中使用 `semaphore.wait()`：协作式线程池线程数有限，阻塞会导致整个池子饿死甚至死锁。

4.8 DispatchWorkItem: 可取消的任务（如防抖）

```

private var searchWorkItem: DispatchWorkItem?

func search(_ text: String) {
    searchWorkItem?.cancel()
    let item = DispatchWorkItem { [weak self] in self?.performSearch(text) }
    searchWorkItem = item
    DispatchQueue.main.asyncAfter(deadline: .now() + 0.3, execute: item)
}

```

`cancel()` 只能阻止尚未开始的 work item 执行；已经在跑的需要自己检查 `item.isCancelled`。

4.9 DispatchSource: 系统事件 / 高精度定时器

```

let timer = DispatchSource.makeTimerSource(queue: .global(qos: .utility))
timer.schedule(deadline: .now(), repeating: .seconds(1))

```

```
timer.setEventHandler { print("tick") }
timer.resume()
// 不依赖 RunLoop，滚动时也照常触发；需持有 timer 引用，否则会被释放
```

还可监听文件变化（`makeFileSystemObjectSource`）、进程信号、内存压力等。

4.10 DispatchQueue.concurrentPerform：并行循环

```
DispatchQueue.concurrentPerform(iterations: images.count) { index in
    thumbnails[index] = resize(images[index]) // 需要保证写入线程安全
}
// 同步调用：所有迭代完成后才返回；会自动按 CPU 核心数并行
```

4.11 sync vs async 小结

	sync	async
调用方	阻塞等待 block 执行完	立即返回
常见用途	从串行队列安全读取值	提交工作
风险	对当前所在的串行队列 sync → 死锁	捕获 <code>self</code> 可能延长生命周期

5. OperationQueue

5.1 是什么

基于 GCD 的面向对象封装。相比 GCD 多了：依赖关系、取消、最大并发数、KVO 状态观察、暂停/恢复。

```
let queue = OperationQueue()
queue.maxConcurrentOperationCount = 2 // 1 即串行队列
queue.qualityOfService = .utility

let download = BlockOperation { downloadFile() }
let unzip     = BlockOperation { unzipFile() }
let save      = BlockOperation { saveToDB() }

unzip.addDependency(download) // 依赖：download 完成后才执行 unzip
save.addDependency(unzip)

queue.addOperations([download, unzip, save], waitUntilFinished: false)

OperationQueue.main.addOperation { updateUI() } // 主队列

queue.cancelAllOperations() // 取消（已在运行的需自行检查 isCancelled）
queue.isSuspended = true // 暂停调度新的 operation
```

5.2 何时使用

- 有复杂依赖图、需要取消/暂停、需要限制并发数的任务（如批量下载、图片处理流水线）。

- 新代码中，大多数场景可以用 `TaskGroup` + 结构化并发替代；但需要"暂停整个队列"运行时修改并发数"时，`OperationQueue` 仍然方便。

6. Swift Concurrency

Swift 5.5 引入、Swift 6 强制数据竞争安全的语言级并发模型。核心理念：**结构化并发 + actor 隔离 + 编译期检查**。

6.1 async / await

```
func fetchUser(id: Int) async throws -> User {
    let (data, _) = try await URLSession.shared.data(from: url(for: id))
    return try JSONDecoder().decode(User.self, from: data)
}
```

- `await` 是潜在挂起点：任务让出线程，等结果返回后再恢复，恢复时可能在另一个线程（但会回到正确的 actor 上）。
- 挂起不会阻塞线程，因此主线程上 `await` 不会卡 UI。

6.2 协作式线程池（Cooperative Thread Pool）

- 非主 actor 的任务默认跑在全局并发执行器上，底层是一个线程数 $\approx$ CPU 核心数的线程池。
- 每个线程依靠任务在 `await` 处主动让出来切换工作（"协作式"）。
- **前进保证（forward progress）**：任务不能无限期阻塞线程。因此在 `async` 代码中禁止：`semaphore.wait()`、`group.wait()`、`Thread.sleep`、长时间同步 I/O、`DispatchQueue.sync` 到可能等待异步任务的队列等。
- 对比 GCD：GCD 在线程阻塞时会补充线程（可能线程爆炸）；协作池不会补充线程（阻塞会饿死）。

6.3 Task：非结构化任务

```
// 从同步上下文进入异步世界
Button("刷新") {
    Task { await viewModel.refresh() }
}

// 保存句柄以便取消或获取结果
let task = Task(priority: .userInitiated) { () -> [Item] in
    try await api.fetchItems()
}
let items = try await task.value
task.cancel()
```

`Task { }` 继承调用方的：

继承项	说明
actor 隔离	在 <code>@MainActor</code> 方法里创建 → 闭包在 <code>MainActor</code> 上运行
优先级	继承当前任务优先级
<code>TaskLocal</code> 值	继承
取消状态	不继承——父任务被取消， <code>Task { }</code> 不会自动取消（非结构化）

⚠ 常见误区：在 `@MainActor` 上下文中 `Task { heavyCPUWork() }`，同步的 `heavyCPUWork()` 仍在主线程执行，会卡 UI。

6.4 Task.detached：脱离上下文的任务

```
Task.detached(priority: .background) {
    // 不继承 actor、优先级、TaskLocal
    let report = await generateReport()
    await MainActor.run { self.report = report }
}
```

- 不继承任何上下文，运行在全局并发执行器上。
- 适合：确实需要与当前 actor 完全无关的后台工作。
- 官方建议优先使用结构化并发或 `nonisolated/@concurrent` 函数，`Task.detached` 是最后手段。

6.5 Task.immediate (Swift 6.2, iOS 26+)

```
Task.immediate { @MainActor in
    isLoading = true           // 同步立即执行，不等待下一轮调度
    await load()              // 第一个挂起点之后才变成普通异步任务
}
```

- 普通 `Task {}` 总是入队，稍后执行；`Task.immediate` 会在当前线程立即同步执行，直到第一个真正的挂起点。
- 适合需要“立刻在当前 actor 上更新状态”的场景，避免一帧的延迟。

6.6 async let：并行的固定数量子任务（结构化）

```
func loadDashboard() async throws -> Dashboard {
    async let user    = api.fetchUser()
    async let orders  = api.fetchOrders()
    async let notices = api.fetchNotices()
    // 三个请求并行执行
    return try await Dashboard(user: user, orders: orders, notices: notices)
}
```

- 是子任务：作用域结束前必须完成；若作用域提前退出（如抛错），未 `await` 的子任务会被自动取消并等待。
- 适合数量在编译期已知的并行操作。

6.7 TaskGroup：动态数量的子任务（结构化）

```
func downloadAll(_ urls: [URL]) async throws -> [URL: Data] {
    try await withThrowingTaskGroup(of: (URL, Data).self) { group in
        for url in urls {
            group.addTask {
                let (data, _) = try await URLSession.shared.data(from: url)
                return (url, data)
            }
        }
        var result: [URL: Data] = [:]
        for try await (url, data) in group { // 按完成顺序收集

```

```

        result[url] = data
    }
    return result
}

```

限制并发数（替代 Semaphore）：

```

await withTaskGroup(of: Void.self) { group in
    let maxConcurrent = 3
    var iterator = items.makeIterator()

    for _ in 0..

```

变体	说明
<code>withTaskGroup</code>	子任务不抛错
<code>withThrowingTaskGroup</code>	子任务可抛错；任一错误被抛出时，其余子任务被取消
<code>withDiscardingTaskGroup</code> (iOS 17+)	不收集结果，子任务完成后立即释放，适合长期运行的"服务器循环"

6.8 取消（Cancellation）

Swift 的取消是协作式的：`cancel()` 只是设置一个标志，任务需要自己检查。

```

func processAll(_ items: [Item]) async throws {
    for item in items {
        try Task.checkCancellation() // 已取消则抛 CancellationError
        // 或: if Task.isCancelled { return }
        await process(item)
    }
}

// 取消时需要立即执行清理（如取消底层回调式请求）
try await withTaskCancellationHandler {
    try await legacyDownload()
} onCancel: {
    legacyRequest.cancel() // 可能在任意线程被调用
}

```

- 结构化子任务（`async let`、`TaskGroup`）会随父任务一起取消。
- `URLSession.data(from:)`、`Task.sleep` 等系统 API 会自动响应取消。
- SwiftUI 的 `.task {}` 在视图消失时自动取消。

6.9 优先级与睡眠/让出

```
Task(priority: .background) { ... }
Task.currentPriority

try await Task.sleep(for: .seconds(1)) // 挂起，不阻塞线程；会响应取消
await Task.yield()                     // 主动让出，给其他任务执行机会（长 CPU 循环中
使用）
```

`TaskPriority: .high(= .userInitiated)`、`.medium`、`.low(= .utility)`、`.background`。系统会做**优先级提升**（priority escalation）：高优先级任务等待低优先级任务时，后者会被提升。

6.10 TaskLocal: 任务级"上下文变量"

```
enum RequestContext {
    @TaskLocal static var traceID: String = "none"
}

await RequestContext.$traceID.withValue("abc-123") {
    await handle() // handle 及其子任务（含 Task {}）都能读到 "abc-123"
}
```

类似线程局部存储，但绑定在任务树上。适合日志追踪 ID 等。

6.11 actor: 保护可变状态

```
actor ImageCache {
    private var cache: [URL: UIImage] = [:]

    func image(for url: URL) -> UIImage? { cache[url] }

    func load(_ url: URL) async throws -> UIImage {
        if let cached = cache[url] { return cached }
        let (data, _) = try await URLSession.shared.data(from: url)
        let image = UIImage(data: data)!
        cache[url] = image // ⚠ 见下方"重入"
        return image
    }
}

let image = try await cache.load(url) // 外部访问必须 await
```

- actor 保证同一时间只有一个任务在访问其内部状态（串行执行），从而消除数据竞争。
- **可重入（reentrancy）**：在 `await` 处，actor 可以去执行其他调用。所以 `await` 之后，之前读取的状态可能已经变化。上例中两个并发调用会重复下载同一 URL；解决方式是缓存"进行中的 Task"：

```
actor ImageLoader {
    private var tasks: [URL: Task<UIImage, Error>] = [:]

    func load(_ url: URL) async throws -> UIImage {
```

```

        if let existing = tasks[url] { return try await existing.value }
        let task = Task { try await download(url) }
        tasks[url] = task
        return try await task.value
    }
}

```

- **nonisolated**: 标记不访问隔离状态的成员，可在外部同步调用。
- **isolated** 参数: `func f(_ a: isolated MyActor)` 让函数运行在参数 actor 上。
- **#isolation**: 获取调用方的隔离，常用于 `isolation: isolated (any Actor)? = #isolation` 参数，让通用 `async` 函数在调用方的 actor 上执行。

6.12 MainActor 与全局 actor

@MainActor 是一个**全局 actor**，其执行器即主线程/主队列。被标注的类型、函数、属性、闭包都被隔离到主线程。

```

@MainActor
@Observable
final class ProfileViewModel {
    var user: User?
    var isLoading = false

    func load() async {
        isLoading = true
        defer { isLoading = false }
        user = try? await api.fetchUser() // 后台执行，返回后自动回到 MainActor
    }
}

```

切换到主 actor 的几种方式：

```

await MainActor.run { self.title = "Done" } // 在主 actor 上执行一个闭包
Task { @MainActor in model.refresh() }      // 新建一个主 actor 任务（总是入队）
MainActor.assumeIsolated { model.update() }  // 断言已在主线程（不在则崩溃），用于旧回调

```

SwiftUI 相关：

- **View** 协议整体是 **@MainActor**（iOS 18 SDK 起；之前仅 **body**）。
- **.task {}** 闭包继承 View 的隔离，即在 MainActor 上运行。

自定义全局 actor：

```

@globalActor
actor DatabaseActor {
    static let shared = DatabaseActor()
}

@DatabaseActor
final class DatabaseStore {
    func save(_ item: Item) { ... } // 所有成员都串行在 DatabaseActor 上
}

```

适合：把一组分散在多个类型中的操作统一到同一个串行隔离域。

6.13 Sendable：跨隔离域传值的安全标记

- **Sendable** 表示值可以安全地在并发域之间传递。
 - 值类型（成员都是 Sendable 的 struct/enum）自动满足。
 - **final class** 且所有属性为 **let** 且是 Sendable → 可声明 **Sendable**。
 - actor 天然 Sendable。
 - 内部用锁保护的 class → **@unchecked Sendable**（开发者自行保证）。
- **@Sendable** 闭包：不能捕获可变的非 Sendable 状态。
- **sending** 参数/返回值（Swift 6）：表示值的所有权被“转移”到另一个隔离域，转移后调用方不能再使用，从而允许传递非 Sendable 值。

6.14 Continuation：桥接回调式 API

```

func fetchLocation() async throws -> CLLocation {
    try await withCheckedThrowingContinuation { continuation in
        legacyLocationManager.requestLocation { result in
            switch result {
            case .success(let location): continuation.resume(returning: location)
            case .failure(let error): continuation.resume(throwing: error)
            }
        }
    }
}

```

- 规则：必须且只能 **resume** 一次。**Checked** 版本会在违反时给出运行时警告/崩溃；**Unsafe** 版本无检查、性能略好。

6.15 AsyncSequence / AsyncStream：异步序列

```

// 消费
for try await line in url.lines { print(line) }
for await notification in NotificationCenter.default.notifications(named:
    .NSCalendarDayChanged) { ... }

// 把回调/委托转为 AsyncStream
func locationUpdates() -> AsyncStream<CLLocation> {
    AsyncStream { continuation in
        let delegate = LocationDelegate { continuation.yield($0) }
        continuation.onTermination = { _ in delegate.stop() }
    }
}

```

```

        delegate.start()
    }
}

// iOS 17+ 便捷构造
let (stream, continuation) = AsyncStream.makeStream(of: Int.self)

```

- 是 Combine **Publisher** 在 Swift Concurrency 中的对应物（拉取式 vs 推送式）。
- **for await** 循环会随任务取消而结束。

6.16 Executor（执行器）

- 每个 actor 有一个**串行执行器**；MainActor 的执行器绑定主线程。
- 非隔离的 async 代码运行在**全局并发执行器**（协作池）上。
- 高级特性：自定义 **SerialExecutor**（让 actor 跑在指定的 DispatchQueue 上，便于与旧代码共享队列）、**TaskExecutor**（SE-0417, **Task(executorPreference:)**）。一般 App 开发很少需要。

6.17 Swift 6 / 6.2 的重要变化

特性	说明
Swift 6 语言模式	数据竞争由 编译错误 而不是警告报告
区域隔离（Region-based isolation, SE-0414）	编译器能证明值未被共享时，允许传递非 Sendable 值
默认 actor 隔离（SE-0466, Swift 6.2）	模块可设置 <i>Default Actor Isolation = MainActor</i> ，未标注代码默认 @MainActor 。Xcode 26 新 App 工程默认开启
nonisolated(nonsending) （SE-0461）	nonisolated 的 async 函数默认在调用方的 actor 上执行（开启 NonisolatedNonsendingByDefault 后），不再自动跳到后台
@concurrent	显式声明"这个 async 函数要在全局并发执行器（后台）上执行"
Task.immediate （SE-0472）	同步开始执行，直到第一个挂起点
isolated deinit （SE-0371）	actor 隔离类型的 deinit 可以在其 actor 上运行
Approachable Concurrency	Xcode 26 的构建设置，一次性开启上述若干"更易用"的特性

在 Swift 6.2 + 默认 MainActor 隔离的工程中，推荐写法：

```

// 默认就在 MainActor 上
final class PhotoViewModel {
    var photo: UIImage?

    func load(_ url: URL) async {
        photo = await Self.decode(url) // 跳到后台解码，回来后更新 UI
    }

    @concurrent
    nonisolated static func decode(_ url: URL) async -> UIImage? {
        // 在后台线程池执行，不阻塞主线程
        guard let data = try? Data(contentsOf: url) else { return nil }
        return UIImage(data: data)?.preparingForDisplay()
    }
}

```

```
}  
}
```

7. 锁与同步原语

当状态需要被同步访问（不能 `await`）时，使用锁。

原语	最低版本	特点
<code>NSLock</code>	—	简单互斥锁， <code>lock.withLock { }</code>
<code>NSRecursiveLock</code>	—	同一线程可重复加锁
<code>NSCondition</code>	—	条件变量
<code>os_unfair_lock</code>	—	最快的底层锁，但在 Swift 中直接用不安全（需固定内存地址）
<code>OSAllocatedUnfairLock</code>	iOS 16	<code>os_unfair_lock</code> 的安全 Swift 封装，可携带受保护状态
<code>Mutex</code> （Synchronization 框架）	iOS 18	Swift 原生互斥锁， <code>~Copyable</code> ，本身 <code>Sendable</code>
<code>Atomic</code> （Synchronization 框架）	iOS 18	无锁原子操作（计数器、标志位）
串行 <code>DispatchQueue</code>	—	用队列做锁（ <code>sync</code> 读、 <code>async</code> 写）
<code>actor</code>	iOS 13	异步互斥，访问需 <code>await</code>

```
import Synchronization  
  
final class Counter: Sendable {  
    private let state = Mutex(0)  
    func increment() { state.withLock { $0 += 1 } }  
    var value: Int { state.withLock { $0 } }  
}  
  
final class Flag: Sendable {  
    private let done = Atomic(false)  
    func markDone() { done.store(true, ordering: .releasing) }  
    var isDone: Bool { done.load(ordering: .acquiring) }  
}
```

actor vs 锁：

	actor	Mutex / 锁
访问方式	异步（ <code>await</code> ）	同步
等待方式	挂起，不占线程	阻塞线程（应该非常短）
可重入问题	有（ <code>await</code> 后状态可能变）	无（临界区内不能 <code>await</code> ）
适合	包含异步操作的复杂状态	小而快的同步临界区；需要在同步代码中访问

⚠ 不要在持有锁时 `await`（`withLock` 闭包是同步的，编译器也会阻止）。

8. Combine 中的 Scheduler

```

publisher
    .subscribe(on: DispatchQueue.global()) // 影响上游订阅/请求发生的线程
    .map { heavyTransform($0) }
    .receive(on: DispatchQueue.main) // 影响下游接收值的线程
    .sink { self.value = $0 }

```

Scheduler	行为
<code>DispatchQueue.main</code>	主线程，滚动时照常执行
<code>RunLoop.main</code>	主线程， <code>.default</code> 模式，滚动时推迟
<code>OperationQueue.main</code>	主线程
<code>DispatchQueue.global()</code> / 自定义队列	后台
<code>ImmediateScheduler.shared</code>	当前线程立即执行（测试中常用）

时间相关操作符（`debounce`、`throttle`、`delay`）也需要指定 Scheduler：

```

$searchText
    .debounce(for: .milliseconds(300), scheduler: DispatchQueue.main)
    .removeDuplicates()
    .sink { query in Task { await self.search(query) } }

```

`@Published` 属性转为 `AsyncSequence`：for await value in `$searchText.values` { ... }。

9. SwiftUI 中与并发相关的 API

9.1 `.task` / `.task(id:)`

```

struct UserView: View {
    let userID: Int
    @State private var user: User?

    var body: some View {
        content
        // 视图出现时启动；视图消失时自动取消；userID 变化时取消旧任务并重启
        .task(id: userID) {
            user = try? await api.fetchUser(id: userID)
        }
    }
}

```

- 闭包继承 View 的 `@MainActor` 隔离。
- 默认优先级 `.userInitiated`，可指定 `.task(priority: .background)` { }。
- 是 SwiftUI 中启动异步工作的首选方式，优于在 `onAppear` 里写 `Task` { }（后者不会自动取消）。

9.2 搜索防抖（用 `.task(id:)` 替代 `DispatchWorkItem` / `Combine debounce`）

```
.task(id: searchText) {
    try? await Task.sleep(for: .milliseconds(300)) // 新输入会取消旧任务, sleep 抛出
取消错误
    guard !Task.isCancelled else { return }
    results = await search(searchText)
}
```

9.3 .refreshable

```
List(items) { ... }
    .refreshable { await viewModel.reload() } // 下拉刷新指示器持续到 async 闭包返回
```

9.4 TimelineView: 按时间表重绘（替代很多 Timer 用法）

```
TimelineView(.periodic(from: .now, by: 1)) { context in
    Text(context.date, style: .time)
}
TimelineView(.animation) { context in ... } // 每帧刷新, 类似 CADisplayLink
```

9.5 Button 中触发异步操作

```
Button("保存") {
    Task { await viewModel.save() } // 继承 MainActor
}
```

9.6 状态更新的线程要求

- `@State`、`@Observable` / `ObservableObject` 驱动 UI 的属性必须在主线程修改。从后台修改 `@Published` 会出现紫色运行时警告。
- 最简单的保证方式：ViewModel 标注 `@MainActor`。

10. 横向对比

10.1 三代并发 API

维度	GCD	OperationQueue	Swift Concurrency
抽象	队列 + block	队列 + Operation 对象	任务 + actor
编译期安全	✗	✗	✓ (Swift 6)
取消	WorkItem (弱)	✓ (需检查 isCancelled)	✓ 协作式, 结构化自动传播
依赖关系	手动 (Group / 嵌套)	✓ addDependency	顺序 await 天然表达
并发数限制	Semaphore	maxConcurrentOperationCount	TaskGroup 手动控制
错误处理	回调中手动传递	手动	throws / try await

维度	GCD	OperationQueue	Swift Concurrency
线程模型	阻塞时补线程（可能爆炸）	同 GCD	协作池，线程数 $\approx$ 核心数
可读性	回调嵌套	中	线性代码
推荐程度	遗留/底层	特定场景	首选

10.2 Task vs Task.detached vs DispatchQueue.global().async let vs TaskGroup

	Task {}	Task.detached {}	DispatchQueue.global().async {}	async let	TaskGroup
结构化	✗	✗	✗	✓	✓
继承 actor 隔离	✓	✗	✗	✗（子任务在并发执行器上；Swift 6.2 nonsending 函数视情况）	✗（addTask 闭包同上）
继承优先级	✓	✗	✗（按 QoS）	✓	✓
继承 TaskLocal	✓	✗	✗	✓	✓
随父任务取消	✗	✗	✗	✓	✓
能否 await 结果	task.value	task.value	✗（需回调）	✓	✓
执行线程	取决于继承的 actor	协作池	GCD 线程池	协作池	协作池
典型用途	同步 $\rightarrow$ 异步入口（按钮点击）	与当前 actor 无关的后台工作	遗留代码后台工作	固定数量并行	动态数量并行

10.3 切到主线程的各种方式

方式	编译期检查	是否总是延后	滚动期间	备注
调用 @MainActor 同步函数（已在主 actor）	✓	否，直接执行	执行	最自然
await 一个 @MainActor 函数（从其他隔离域）	✓	跳转后执行	执行	
await MainActor.run { }	✓	已在主 actor 时不排队	执行	
Task { @MainActor in }	✓	是（入队）	执行	

方式	编译期检查	是否总是延后	滚动期间	备注
<code>Task.immediate { @MainActor in }</code>	✅	已在主 actor 时否	执行	iOS 26+
<code>MainActor.assumeIsolated { }</code>	运行时断言	否	—	不在主线程会崩溃
<code>DispatchQueue.main.async { }</code>	部分（编译器特殊处理）	是	执行	
<code>OperationQueue.main.addOperation { }</code>	❌	是	执行	
<code>RunLoop.main.perform { }</code>	❌	是	默认 mode 下不执行	
<code>.receive(on: RunLoop.main)</code>	❌	是	推迟	Combine

10.4 定时 / 延迟的各种方式

方式	依赖 RunLoop	滚动时	取消	适用
<code>Timer.scheduledTimer / Timer.publish(in: .default)</code>	✅	暂停	<code>invalidate</code>	旧代码
<code>Timer.publish(on: .main, in: .common)</code>	✅	继续	<code>cancel</code> 订阅	SwiftUI <code>onReceive</code>
<code>DispatchQueue.main.asyncAfter</code>	❌	继续	WorkItem	简单延迟
<code>DispatchSourceTimer</code>	❌	继续	<code>cancel()</code>	后台高精度定时
<code>CADisplayLink</code>	✅	取决于 mode	<code>invalidate</code>	逐帧动画
<code>try await Task.sleep(for:)</code>	❌	继续	✅ 随任务取消	async 代码首选
<code>TimelineView</code>	—	—	随视图生命周期	SwiftUI 周期性重绘

11. 执行时机与滚动行为

11.1 执行顺序示例

```
@MainActor
func demo() {
    print("1")
    DispatchQueue.main.async { print("A - DispatchQueue.main.async") }
    Task { @MainActor in      { print("B - Task @MainActor") }
    RunLoop.main.perform      { print("C - RunLoop.main.perform") }
    OperationQueue.main.addOperation { print("D - OperationQueue.main") }
    Task.detached              { print("E - detached (后台线程)") }
    syncMainActorFunc()        // 直接执行，不排队
    print("3")
}
```

```
}

@MainActor func syncMainActorFunc() { print("2") }
```

- 1、2、3 一定按顺序最先输出（主线程上的同步代码）。
- A/B/C/D 都在 `demo()` 返回之后执行，它们之间的相对顺序不应被依赖。
- E 在后台线程，可能在任何时刻输出（甚至在 2 之前）。

11.2 滚动期间的差异

```
// 用户正在拖动 List 时:
publisher.receive(on: DispatchQueue.main)           // ✅ 持续更新
publisher.receive(on: RunLoop.main)                 // ❌ 推迟到滚动结束
Timer.publish(every: 1, on: .main, in: .default)    // ❌ 滚动时暂停
Timer.publish(every: 1, on: .main, in: .common)     // ✅ 滚动时继续
Task { @MainActor in ... }                          // ✅ 执行
```

12. GCD → Swift Concurrency 迁移对照

GCD / 旧写法	Swift Concurrency 写法
<code>DispatchQueue.main.async { }</code>	类型/函数标 <code>@MainActor</code> ; 或 <code>await MainActor.run { }</code>
<code>DispatchQueue.global().async { }</code>	<code>@concurrent / nonisolated async</code> 函数; 或 <code>Task.detached</code>
completion handler	<code>async</code> 函数; 旧 API 用 <code>withCheckedContinuation</code> 包装
<code>DispatchGroup + notify</code>	<code>async let / withTaskGroup</code>
<code>DispatchSemaphore</code> 限流	<code>TaskGroup</code> 控制在途任务数
串行队列保护状态	<code>actor</code> 或 <code>Mutex</code>
并发队列 + barrier	<code>actor</code> (或读多写少时 <code>Mutex / OSAllocatedUnfairLock</code>)
<code>asyncAfter</code>	<code>try await Task.sleep(for:)</code>
<code>DispatchWorkItem</code> 防抖	<code>.task(id:) + Task.sleep</code>
<code>OperationQueue</code> 依赖	按顺序 <code>await</code>
<code>Thread.isMainThread</code> 检查	<code>@MainActor</code> 静态保证; <code>MainActor.assumeIsolated</code>
委托/通知回调流	<code>AsyncStream /</code> <code>NotificationCenter.notifications(named:)</code>
Combine <code>Publisher</code>	<code>AsyncSequence (publisher.values)</code>

13. 常见陷阱

1. 在 `@MainActor` 里用 `Task { }` 跑 CPU 密集型工作 → 仍在主线程，卡 UI。改用 `@concurrent / nonisolated` 函数或 `Task.detached`。
2. 在 `async` 上下文中阻塞 (`semaphore.wait()`、`group.wait()`、`Thread.sleep`、`DispatchQueue.sync` 等待异步结果) → 协作池饿死/死锁。

3. 主线程 `DispatchQueue.main.sync`；或对当前所在的串行队列 `sync` → 死锁。
4. **GCD 线程爆炸**：大量阻塞任务丢进 `global` 队列。
5. **actor 重入**：`await` 之后假设状态没变 → 重复下载、状态错乱。`await` 后重新检查状态，或缓存进行中的 `Task`。
6. 忘记取消：`onAppear { Task { } }` 不会随视图消失而取消；用 `.task { }`。保存的 `Task` 句柄在 `deinit / onDisappear` 中 `cancel()`。
7. 非结构化任务的 **self 强引用**：`Task { await self.longRunning() }` 会让 `self` 存活到任务结束。长任务需配合取消或 `[weak self]`。
8. **Continuation 未 resume 或 resume 两次** → 任务永远挂起 / 崩溃。
9. 从后台线程修改 `@Published / @Observable` 驱动 UI 的属性 → 运行时警告、UI 不一致。
10. `receive(on: RunLoop.main)` 导致滚动时 UI 不更新；`Timer` 用了 `.default` mode 导致滚动时停表。
11. 依赖各种"切到主线程"方式之间的执行顺序。
12. **@unchecked Sendable 滥用**：只是让编译器闭嘴，不会让代码真正线程安全。
13. `global` 队列上用 `barrier`：无效。

14. 选择指南

需要做什么？

更新 UI / 持有 UI 状态	→	@MainActor (ViewModel 整体标注)
视图出现时加载数据	→	.task { } / .task(id:) { }
按钮点击触发异步操作	→	Task { await ... }
耗时的 CPU 工作（解码、解析）	→	@concurrent / nonisolated async 函数 (遗留代码: DispatchQueue.global)
多个请求并行		
└ 数量固定	→	async let
└ 数量动态 / 需限流	→	withTaskGroup
共享可变状态		
└ 内部有 async 操作	→	actor
└ 小而快、需同步访问	→	Mutex / OSAllocatedUnfairLock / Atomic
包装回调式 API	→	withCheckedContinuation
包装委托 / 事件流	→	AsyncStream
延迟 / 防抖	→	Task.sleep / .task(id:)
周期性 UI 刷新	→	TimelineView / Timer.publish(in: .common)
逐帧回调	→	TimelineView(.animation) / CADisplayLink
复杂依赖图、可暂停的任务队列	→	OperationQueue
对接旧 SDK 的回调切回主线程	→	DispatchQueue.main.async / MainActor.assumeIsolated
└ 需要专用长期线程 + RunLoop	→	Thread (极少)

15. 一句话总结

概念	一句话
Thread	最底层的线程，几乎不用直接操作。

概念	一句话
RunLoop.main	主线程的事件循环，用于 Timer/DisplayLink，有 Mode，滚动时行为不同。
DispatchQueue.main	GCD 主串行队列，"把 block 排到主线程执行"，适合遗留回调。
DispatchQueue.global	GCD 共享并发后台队列，按 QoS 分级，阻塞过多会线程爆炸。
自定义 DispatchQueue	串行队列做锁，并发队列 + barrier 做读写锁。
DispatchGroup / Semaphore / WorkItem / Source	等待一组任务 / 限流 / 可取消任务 / 系统事件与定时器。
OperationQueue	面向对象的任务队列，支持依赖、取消、并发数控制。
async/await	以线性代码书写异步逻辑，挂起不阻塞线程。
Task	非结构化任务，继承 actor/优先级/TaskLocal，不随父任务取消。
Task.detached	什么都不继承的非结构化任务，最后手段。
Task.immediate	立即在当前上下文同步开始执行的任务（Swift 6.2）。
async let / TaskGroup	结构化子任务，自动等待、自动取消传播。
actor	串行访问的状态容器，消除数据竞争，注意重入。
MainActor	主线程的全局 actor，编译器保证 UI 代码在主线程，SwiftUI 首选。
Sendable	跨并发域传值的安全证明。
Continuation / AsyncStream	把回调 / 事件流桥接到 async 世界。
Mutex / Atomic / 锁	同步的小临界区保护，不能跨 await。
.task (SwiftUI)	与视图生命周期绑定、自动取消的异步入口。